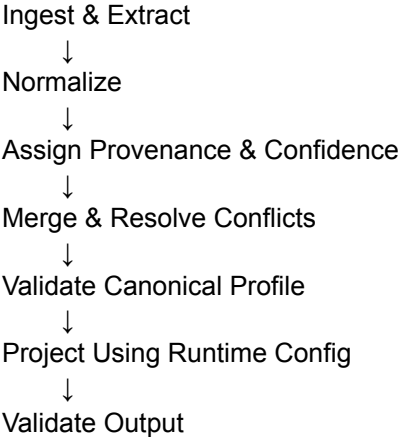


Candidate Data Transformer – Technical Design

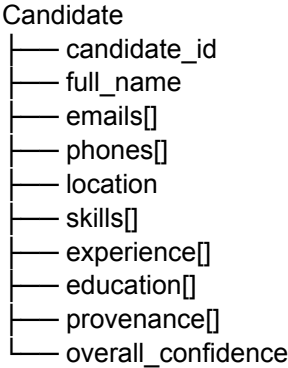
Author: Nikita Gupta

Role: Backend/Data Engineer Intern Eightfold Assignment

1. Pipeline Architecture



2. Canonical Profile Schema



A deterministic candidate ID is generated from normalized identity fields to ensure repeatable output.

3. Merge & Conflict Resolution

Identity Matching

Build a graph connecting records that share normalized emails or phone numbers, then merge each connected component into a canonical profile.

Confidence

- Recruiter CSV: **0.95**
- Resume Parser: **0.75**
- Multi-source agreement increases confidence (capped at 0.99).

4. Runtime Projection Layer

Runtime JSON configuration reshapes the canonical profile into consumer-specific output without changing the core pipeline.

Supported features:

- Select output fields
- Rename fields
- Field mapping
- Toggle provenance/confidence
- Missing value handling (null, omit, error)

5. Edge Cases

- Malformed CSV rows skipped with warnings.
- Invalid emails/phones removed.
- Duplicate skills merged after normalization.
- Missing sources handled gracefully.
- Unknown values returned as null.

6. Scalability

Normalization operates in linear time. Identity matching constructs a graph using normalized email and phone keys and merges connected components efficiently, making the pipeline suitable for large candidate datasets.

7. Out of Scope

- Fuzzy name matching
- OCR for scanned resumes
- ML/NLP-based extraction
- Cross-language skill normalization
- LinkedIn scraping